UNIDAD CENTRAL DEL VALLE DEL CAUCA
FACULTAD DE INGENIERÍA
ARQUITECTURA DE SOFTWARE
PARCIAL II - MICROFRONTENDS & CLEAN ARCHITECTURE
DOCENTE: JESUS DAVID MEJIA VERGARA

DESCRIPCIÓN

En este parcial, los estudiantes deberán diseñar e implementar una solución de software distribuida basada en una arquitectura de microfrontends utilizando Angular y Module Federation, integrada con una arquitectura de microservicios desarrollados en Node.js o Spring Boot.

El propósito es que los estudiantes apliquen los conceptos y principios de arquitectura de software estudiados, comprendiendo cómo dividir un sistema en componentes independientes, modulares y especializados, capaces de integrarse para ofrecer una solución funcional. Para ello, deberán implementar Arquitectura Limpia (Clean Architecture) en cada microfrontend y microservicio, promoviendo la separación de responsabilidades, la independencia entre capas y la facilidad de mantenimiento y evolución del software.

La solución contará con un host encargado de integrar y gestionar la navegación entre los diferentes microfrontends, mientras que cada microfrontend deberá comunicarse con un microservicio específico mediante una API. De esta manera, se busca que los estudiantes fortalezcan sus competencias en diseño arquitectónico, integración de componentes distribuidos, comunicación entre sistemas y aplicación de buenas prácticas de desarrollo, comprendiendo los retos y beneficios de construir sistemas escalables, desacoplados y mantenibles.

INDICACIONES GENERALES

- El parcial se desarrollará en grupos de tres (3) estudiantes, debido a la extensión y complejidad de la prueba.
- Todos los proyectos deberán estar alojados en un único monorepositorio en GitHub.
- Se deberá aplicar el estándar de commits establecido para el curso.
- Se verificará la participación de cada estudiante mediante el historial de commits.
- Todas las aplicaciones, tanto de frontend como de backend, deberán contar con documentación técnica.

- Los microfrontends deberán implementar pruebas unitarias y alcanzar un nivel de cobertura superior al 90%.

- Los microservicios deberán documentar sus APIs mediante Swagger.

- Todas las aplicaciones deberán ejecutarse correctamente desde la consola.

- Las aplicaciones que no puedan ejecutarse correctamente recibirán una calificación de cero (0,0) en el componente correspondiente.

- Antes de la entrega, se deberá verificar el funcionamiento individual e integrado de todos los componentes.

- El repositorio deberá incluir un archivo README.md con las instrucciones de ejecución y configuración.

- El README deberá especificar los puertos de ejecución del host, los microfrontends y los microservicios.

- Se deberá documentar qué microservicio consume cada microfrontend y el puerto correspondiente.

- Las conexiones entre microfrontends y microservicios deberán funcionar de acuerdo con la arquitectura definida y permitir su verificación mediante las solicitudes de red.

- Durante el curso se ha suministrado la configuración y la documentación de referencia necesarias para desarrollar la prueba.

- Las implementaciones deberán ser coherentes con las tecnologías y los conceptos trabajados durante el curso.

- Si se identifican implementaciones sospechosas o que incorporen mecanismos no abordados en clase, el docente podrá solicitar una sustentación técnica a uno de los integrantes del grupo (**Principalmente si afectan Clean Architecture**, no hay problema si se ayudan de la IA para generar el HTML y SCSS de las páginas).

- La calificación obtenida en dicha sustentación será aplicada a todos los integrantes del grupo. Por tanto, cada estudiante deberá comprender y justificar las implementaciones realizadas.

- Se deberá enviar el enlace del repositorio en un archivo PDF como entrega.

PUNTO 1 [2.5] DESARROLLO DE MICROFRONTENDS

Se deberá implementar una arquitectura de microfrontends utilizando Angular y Module Federation, compuesta por un host y tres microfrontends independientes, integrados correctamente mediante la configuración correspondiente. El host deberá contar con un navbar funcional que permita navegar entre los diferentes microfrontends.

Cada microfrontend deberá aplicar los principios de Arquitectura Limpia (Clean Architecture), organizando sus componentes y separando las responsabilidades por capas.

Cada microfrontend deberá implementar tres casos de uso, para un total de nueve casos de uso en el frontend. Estos deberán gestionar las solicitudes de información mediante los casos de uso correspondientes, utilizando API Gateways o repositorios para comunicarse con el microservicio asignado a cada microfrontend.

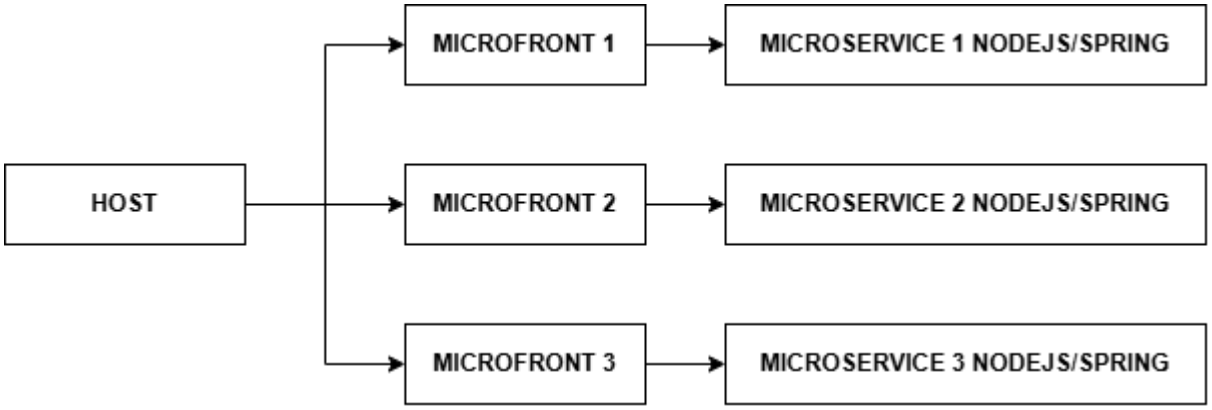
PUNTO 2 [2.5] DESARROLLO DE MICROSERVICIOS

Se deberán implementar tres microservicios independientes, cada uno asociado a un microfrontend específico. Los servicios podrán desarrollarse utilizando Node.js y Spring Boot, garantizando la comunicación mediante APIs y aplicando los principios de Arquitectura Limpia.

Cada microservicio deberá implementar tres casos de uso, para un total de nueve casos de uso en el backend. Será obligatorio desarrollar al menos un microservicio en Node.js y otro en Spring Boot; el tercer microservicio podrá desarrollarse libremente con cualquiera de las dos tecnologías.

La solución deberá garantizar la correcta integración entre los microfrontends y sus respectivos microservicios, manteniendo la separación de responsabilidades y la independencia de cada componente.

ESTRUCTURA GLOBAL DEL PROYECTO



ESTRUCTURA DE CADA MICROFRONTEND CON SU MICROSERVICIO

